

Video Transcript

Video 12: OAuth2 Setup Example (08:18)

I want to show you a quick demo of how we lock down our APIs. This is going to be using OAuth2, and in the implementation is Okta's, a company called Okta. So, let's take a look. What I've got here is I've got a web server, an express web server that's fronting a database. I'm using a simple database, lowdb because that writes into a file, and we can actually see quite easily what we've done in the database. And the file it writes to is db.json. So, what I'm showing here is, I've got a posts array and into that post I'm putting objects with an id.

These are going to be book titles and whether it's published or not, okay. And so, I've got these in there already. If I don't actually put an id, it's going to take daytime and create an id out of that daytime okay. So, what I want to make sure is that we lock down any calls to the express server and in particularly certain routes, okay? So, we want to make sure that if somebody calls, for example, this get on posts, they need permission.

And the way this is done is that we're going to use this middleware, this Okta middleware that's going to require that we have the right permissions. So, if there's a get, the get needs a token in order to be able to call this method, this post method, okay. And let's take a look at, we're going to use this client to make calls to that API. And the client to make those calls, it's going to be calling this lowClientAuth.js, and what we need is we need to provide certain information. So, we need to be registered, if you like, with the OAuth2 server. And so, this is a server that's sitting in the cloud that I've already set up.

I've set it up to process any requests to this API. And I already have a client id and a client secret, and I know the issuer of that. And we're getting those from a .environment file. So, if we look at the .environment file, I've put into that, this is the CLIENT_ID that I got from the authorization server. I've got a CLIENT_SECRET, and this is the issuer, and the scope is for something called parts manager, and that's one that I've set up. So, we're going to run this from a terminal window. I'm in the Authlow db, and I'm going to run node low db.js. And that's going to spin up the server listening on 3000 and it's populated and it's listing out some things just to show that it's working.

So, now it's listening on 3000. So, let me go to localhost, and if I hit 3000, and I need to hit posts and that should. It won't let me hit it because I need an authorization, okay, I'm not authorized to just hit it like that. I need to go and get a token. So, let's go back and go back to this which, so in this client there are two parts. One is we need to go and hit the authorization server with our credentials.

So, this is particularly our CLIENT_ID and our CLIENT_SECRET. We do that in this send API request, so it's an asynchronous method. So, we try first this where we're packaging up our CLIENT_ID and CLIENT_SECRET into a token. We're going to get back an authorization if everything is right. And so, we make a request and we're making it to this URI, so, it's just the same as a URL. And we're going to post this information, our CLIENT_ID and CLIENT_SECRET, and as to get back a token. With that token, so, here I'm just going to list out that authorization.

Now, we make another request to the API that we want to hit, and if we've got the right token, that API will let us through. So, let's take a look at the API, and in the API, you see that we've got this authorization middleware and we're going to use that. So, this is going to stand between us and the API.

This OAuth2 and will stop us as we saw above, it stopped us hitting, for example, posts. But if we've got the right authorization token that we'll be able to hit this API. So, let's actually run this now. So, this would, for example, let's start off by just hitting posts. So, we're calling our client that will call over here to this API. And, we're just asking to list posts, we've just list out everything that's in the database. So, there it is. And, what I'm printing out here is the authorization token, the access token that came back. And here is the data that's being returned from that API. So, that's how the authorization works. We can do a little more complicated one. Let me clear that. Let's suppose we want to add a title.

So, here I'm calling posts, but I'm calling the POST method and I'm putting in a title. I put in "Pretty Deadly Cooking". Let's just change it in case it's in the database already. Let's call it "Fairly Deadly Cooking", okay? And we need to post that now. Let's see if it does. Yep, you see that it got posted over here "Fairly Deadly Cooking". So, and again, we've got our authorization token that came back and then it was authorized, and we could hit that API. So, it's very similar to JSON web tokens that we've used before. But in this case, Okta had a server in the cloud that was doing the approval of those web tokens. Okay.